Link to copilot chat:
https://copilot.cloud.microsoft/chat/share/eyJzaGFyZUlkIjoiYzM2MjNmN2UtODZhNy00ZGQ1LTgwODAtM2E0N2FkMThkOTE4IiwiY29udmVyc2F0aW9uSWQiOiIzYmI1YTg0ZS04NWUxLTQzNmMtODE5NC1iYTUxNDU3YzE0NzQifQ%3D%3D

Input 1:

```
String choice = sc.nextLine();
while (choice != "Y" && choice != "N") {
        System.out.println(choice + " is invalid");
        System.out.print("Please enter either 'Y' or 'N': ");
        choice = sc.nextLine();
}
```

why does my java code keep entering the while loop?

Output 1:

Your loop keeps running because in Java, != compares **object references**, not the actual text inside strings.

while (choice != "Y" && choice != "N")

Even if the user types "Y", the String returned by sc.nextLine() is usually a different object than the string literal "Y", so choice != "Y" evaluates to true.

Fix

Use equals() to compare string contents:

```
String choice = sc.nextLine();
while (!choice.equals("Y") && !choice.equals("N")) {
  System.out.println(choice + " is invalid");
  System.out.print("Please enter either 'Y' or 'N': ");
  choice = sc.nextLine();
}
```

Even safer (handles lowercase input)

```
String choice = sc.nextLine().trim().toUpperCase();
while (!choice.equals("Y") && !choice.equals("N")) {
  System.out.println(choice + " is invalid");
  System.out.print("Please enter either 'Y' or 'N': ");
  choice = sc.nextLine().trim().toUpperCase();
}
```

Why == / != doesn't work for Strings

```
String a = new String("Y");
String b = "Y";
System.out.println(a == b);     // false (different objects)
System.out.println(a.equals(b)); // true  (same text)
```

For strings in Java, use:

- equals() → compare contents ✅
- == or != → compare whether they are the exact same object in memory ❌

That's why your loop keeps treating "Y" and "N" as invalid and repeatedly enters the while loop.

Input 2:
int bill = order.getItem().getPrice() * (int) (cust.getBilling()/100);
why does bill return 0

Output 2:
Most likely because of **integer division**.
Consider this expression:
int bill = order.getItem().getPrice() * (int) (cust.getBilling() / 100);
If cust.getBilling() returns an int, then:
cust.getBilling() / 100
uses integer division.
Examples:
50 / 100 // 0
75 / 100 // 0
99 / 100 // 0
100 / 100 // 1
150 / 100 // 1
So if cust.getBilling() is less than 100:
(int)(cust.getBilling() / 100)
becomes:
(int)0
and therefore:
price * 0 = 0
Example
int price = 10;
int billing = 50;
int bill = price * (int)(billing / 100);
// 10 * (int)0
// 10 * 0
// = 0
Fix
If you're trying to calculate a percentage, use floating-point division:
double bill = order.getItem().getPrice() * (cust.getBilling() / 100.0);
or
double bill = order.getItem().getPrice() * ((double) cust.getBilling() / 100);
Debug it

Print the intermediate values:
System.out.println("Price: " + order.getItem().getPrice());
System.out.println("Billing: " + cust.getBilling());
System.out.println("Billing/100: " + (cust.getBilling() / 100));
This will quickly show whether the division is producing 0.
If you tell me the values returned by getPrice() and getBilling(), I can show exactly what the expression evaluates to.

Input 3:
int bill = order.getItem().getPrice() * (cust.getBilling()/100.0);
bill is 0 even though getPrice() and getBilling() return integer

Output 3:
If this line compiles:
int bill = order.getItem().getPrice() * (cust.getBilling() / 100.0);
then something slightly different may be in your actual code, because:
cust.getBilling() / 100.0
produces a double, and multiplying an int by a double produces a double. Java would not allow assigning that directly to an int without a cast.
For example:
int price = 100;
int billing = 50;
double bill = price * (billing / 100.0); // 50.0
Why are you getting 0?
Let's check a few possibilities.
1. getPrice() returns 0
System.out.println(order.getItem().getPrice());
2. getBilling() returns 0
System.out.println(cust.getBilling());
3. You're casting too early
If your actual code is something like:
int bill = order.getItem().getPrice() * (int)(cust.getBilling() / 100.0);
then:
cust.getBilling() = 50
50 / 100.0 = 0.5
(int)0.5 = 0
price * 0 = 0
which explains the result.
4. You're storing the final result as an int
For example:
int bill = (int)(10 * (5 / 100.0));

gives:
10 * 0.05 = 0.5
(int)0.5 = 0
because the fractional part is truncated.

Debug with prints

Print every step:

```
System.out.println("price = " + order.getItem().getPrice());
System.out.println("billing = " + cust.getBilling());
double percentage = cust.getBilling() / 100.0;
System.out.println("percentage = " + percentage);
double bill = order.getItem().getPrice() * percentage;
System.out.println("bill = " + bill);
```

That will immediately show where the zero is coming from.
If you paste the exact values of getPrice() and getBilling(), or the exact line that compiles in your program, I can calculate the result and pinpoint the issue.